

Databases

Project : Designing a Telemetry and UX Database for *Chocolate-Doom*

Fergie Lindsay Díaz, Juan Esteban Ortega, Juan Esteban, Federico Mejia

May 5, 2026

Contents

1	Introduction and context	1
2	Assumptions and requirements	2
2.1	Assumptions	2
2.2	Functional requirements	2
2.3	Non-functional requirements	3
3	Entity-Relation Diagram - Initial Draft	3
3.1	ER Diagram - Explanation	4
4	Relational Schema - Initial Draft	4
4.1	Entities	4
4.2	Entities with attributes	4
4.3	Primary Keys	6
4.4	Foreign Keys	6
4.5	Relationships	7
5	Conclusions	7

1 Introduction and context

This project is about designing and building a database to store and analyze data collected from a modified version of the game *Chocolate-Doom*. The game records detailed information during gameplay, such as the player's position, movement, and status at every moment. In addition to this, players also provide basic personal information and complete a user experience survey.

The main goal in the present report or document of this project is to organize all this data in a structured way so it can be stored, accessed, and analyzed efficiently. To do this, we will design a relational database that connects gameplay data with user and survey information. The database should be able to handle a large amount of data and ensure that everything is consistent and well organized.

Another important part of the project is to run queries that help us understand player behavior. For example, we will look at how players move, how close they are to each other, and whether they cooperate during the game. We will also relate this information to their survey responses to see how their experience connects to their behavior.

Finally, we will follow good practices for data handling, including cleaning and loading the data properly, and considering basic privacy and ethical aspects. Overall, the project aims to combine database design with practical analysis of real gameplay data.

2 Assumptions and requirements

2.1 Assumptions

- Each **User** corresponds to a single real-world participant who has provided informed consent.
- A **User** may have one or more **Player** identities (in-game aliases), but each Player belongs to exactly one User.
- Each **Game** represents a single gameplay session with a defined start and end time.
- Telemetry is recorded at a fine-grained level (per tic) for each Player within a Game.
- Each TelemetryEvent is uniquely identified by $(game_id, player_id, tic)$.
- Each Game is associated with exactly one Episode and one Map.
- Maps are composed of multiple Sectors used for spatial analysis.
- Each participant completes exactly one UX instrument (PENS, GUESS, or BANGS).
- Survey responses are collected after gameplay sessions and are linked to the corresponding User.
- Telemetry data is assumed to be cleanly timestamped and ordered by tic.

2.2 Functional requirements

The next items will show what we assumed from the text, of what the system will need to do:

- Store and manage user demographic information (age, gender, experience level, consent).
- Support multiple player identities per user.
- Record high-frequency gameplay telemetry including position, momentum, orientation, and combat stats.
- Store game session metadata including episode, map, start time, and end time.
- Support spatial grouping of telemetry data by map sectors.
- Ingest raw telemetry logs (TSV format) into a structured relational database.
- Store and manage UX survey instruments (PENS, GUESS, BANGS).
- Store user responses to UX instruments at the question level.
- Support analytical queries on movement, proximity, and player behavior.
- Support aggregation of telemetry data for trajectory and cooperation analysis.

2.3 Non-functional requirements

- The database should handle large telemetry datasets efficiently (at least 20,000 records, but ideally much more as data grows).
- The schema should work well as the number of players, games, and telemetry records increases, without needing major changes.
- Primary keys and foreign keys should be used to keep data consistent across all tables.
- Indexes should be added to improve performance for common queries, especially those involving time sequences and spatial data.
- The design should allow new telemetry fields or new UX instruments to be added without redesigning the whole database.
- Data loading should be repeatable using a clear ETL process so the database can be rebuilt from raw logs if needed.
- User data should be anonymized, and consent must be stored before any data is collected or used.
- If some telemetry records are invalid or broken, they should be logged and skipped without stopping the whole ingestion process.

3 Entity-Relation Diagram - Initial Draft

This section presents the initial conceptual model of the system. It defines the main entities involved in the telemetry and UX data pipeline and illustrates how they interact. The ER diagram serves as the foundation for the relational schema that will be implemented in later stages of the project.

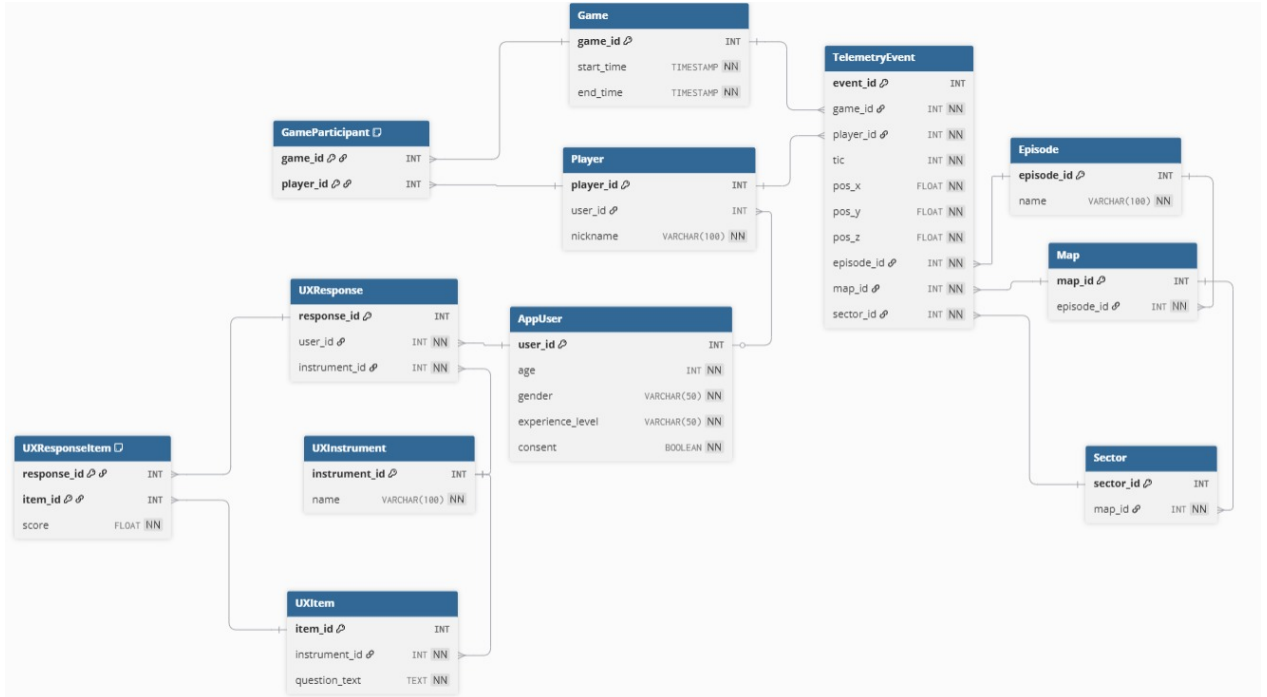


Figure 1: ER Diagram - Initial Draft

3.1 ER Diagram - Explanation

The diagram organizes gameplay telemetry, user information, and survey responses into coherent relationships. It highlights how users are linked to players, how players participate in game sessions, and how each session generates detailed telemetry data. It also integrates UX instruments to connect behavioral data with subjective user experience.

4 Relational Schema - Initial Draft

This section translates the conceptual ER model into a relational schema. It defines the entities, entities with attributes, and keys required to implement the database in a structured and normalized form.

4.1 Entities

This subsection lists the core entities identified in the system. These represent the main objects stored in the database, including users, gameplay sessions, telemetry records, and UX survey components.

- AppUser
- Player
- Game
- GameParticipant
- TelemetryEvent
- Episode
- Map
- Sector
- UXInstrument
- UXItem
- UXResponse
- UXResponseItem

4.2 Entities with attributes

Each entity is defined with its associated attributes, including primary and foreign keys. These attributes capture both gameplay telemetry and user experience data in a structured format suitable for relational storage.

- **User**
 - user_id (PK)
 - age
 - gender
 - experience_level
 - consent
- **Player**

- player_id (PK)
 - user_id (FK, optional)
 - nickname
- **Game**
 - game_id (PK)
 - start_time
 - end_time
- **TelemetryEvent**
 - event_id (PK)
 - game_id (FK)
 - player_id (FK)
 - tic
 - pos_x
 - pos_y
 - pos_z
 - episode
 - map_id
 - sector_id
- **Map**
 - map_id (PK)
 - episode
- **Sector**
 - sector_id (PK)
 - map_id (FK)
- **UXInstrument**
 - instrument_id (PK)
 - name
- **UXItem**
 - item_id (PK)
 - instrument_id (FK)
 - question_text
- **UXResponse**
 - response_id (PK)
 - user_id (FK)
 - instrument_id (FK)
- **UXResponseItem**
 - response_id (FK)
 - item_id (FK)
 - score

4.3 Primary Keys

This subsection defines the unique identifiers for each entity. Primary keys ensure that every record in the database can be uniquely identified and referenced across relationships.

- User: user_id
- Player: player_id
- Game: game_id
- TelemetryEvent: event_id
- Map: map_id
- Sector: sector_id
- UXInstrument: instrument_id
- UXItem: item_id
- UXResponse: response_id
- UXResponseItem: (response_id, item_id)

4.4 Foreign Keys

The way in which we write or show the foreign keys will later help us in the way we can use it in SQL.

Example:

```
FOREIGN KEY (user_id) REFERENCES User(user_id)
```

This subsection defines the relationships between tables using foreign keys. These constraints enforce referential integrity and ensure consistency across users, gameplay sessions, telemetry events, and UX responses.

- **Player:**
user_id → User(user_id)
- **TelemetryEvent:**
game_id → Game(game_id)
player_id → Player(player_id)
map_id → Map(map_id)
sector_id → Sector(sector_id)
- **Sector:**
map_id → Map(map_id)
- **UXItem:**
instrument_id → UXInstrument(instrument_id)
- **UXResponse:**
user_id → User(user_id)
instrument_id → UXInstrument(instrument_id)
- **UXResponseItem:**
response_id → UXResponse(response_id)
item_id → UXItem(item_id)

4.5 Relationships

This subsection summarizes how entities interact within the system. It describes cardinalities and dependencies that support gameplay tracking, spatial analysis, and user experience integration.

- AppUser 1 $\rightarrow$ N Player
- Player $M \leftrightarrow N$ Game (via GameParticipant)
- Game 1 $\rightarrow N$ TelemetryEvent
- Player 1 $\rightarrow N$ TelemetryEvent
- Sector 1 $\rightarrow N$ TelemetryEvent
- Episode 1 $\rightarrow N$ Map
- Map 1 $\rightarrow N$ Sector
- User 1 $\rightarrow N$ UXResponse
- UXInstrument 1 $\rightarrow N$ UXItem
- UXResponse 1 $\rightarrow N$ UXResponseItem
- UXItem 1 $\rightarrow N$ UXResponseItem

5 Conclusions

In this project, we designed a relational database to store and manage gameplay telemetry data together with user information and UX survey responses. By organizing the system into clear tables and relationships, we made sure the data is stored in a structured and scalable way.

A key result of the project is that we were able to connect real-time gameplay data with user experience surveys. This allows us to study not only how players behave in the game, but also how they feel about their experience.

We also defined primary and foreign keys to keep the data consistent and to make queries more efficient. This helps when analyzing player movement, distance between players, and possible cooperation patterns. The design also allows us to easily add new data fields or new survey types in the future without changing the whole database.

Overall, this project shows how relational databases can be used to organize complex game data and support useful analysis in a clear and efficient way.